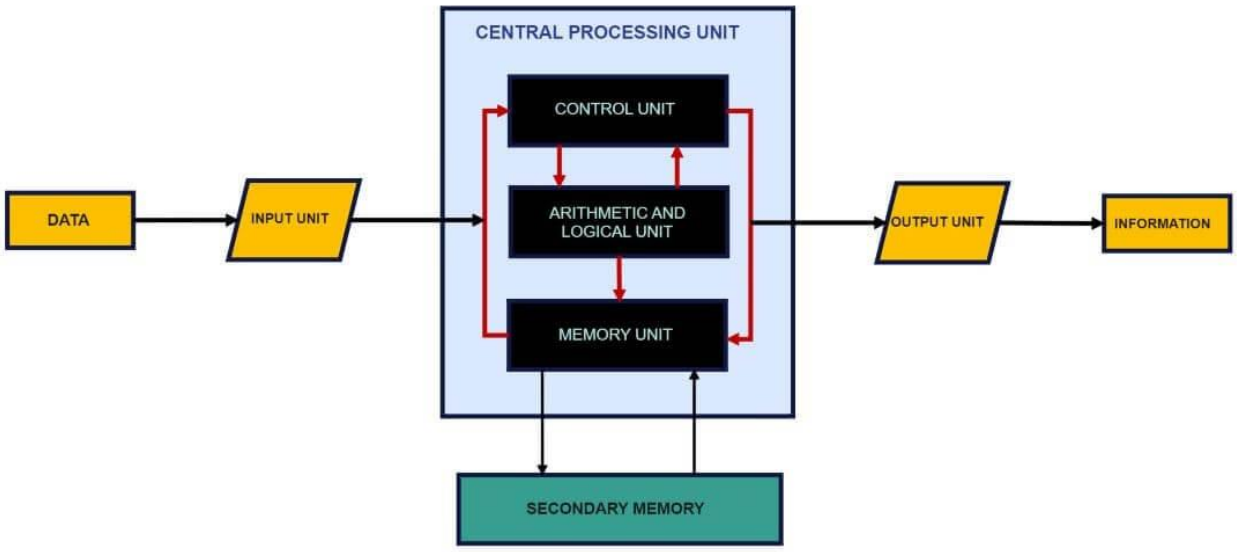
Set -2 Part B

Sketch the internal organization of CPU out with its functionalities and block diagram.



Input

All the data received by the computer goes through the input unit. The input unit comprises different devices like a mouse, keyboard, scanner, etc. In other words, each of these devices acts as a mediator between the users and the computer.

The data that is to be processed is put through the input unit. The computer accepts the raw data in binary form. It then processes the data and produces the desired output.

Major Functions of the Input Unit

The 3 major functions of the input unit are-

- Take the data to be processed by the user.
- Convert the given data into machine-readable form.
- And then, transmit the converted data into the main memory of the computer. The sole purpose is to connect the user and the computer. In addition, this creates easy communication between them.

CPU – Central Processing Unit

Central Processing Unit or the CPU, is the brain of the computer. It works the same way a human brain works. As the brain controls all human activities, similarly the CPU controls all the tasks.

Moreover, the CPU conducts all the arithmetical and logical operations in the computer.

Now the CPU comprises of two units, namely – ALU (Arithmetic Logic Unit) and CU (Control Unit). Both of these units work in sync. The CPU processes the data as a whole.

Let us see what particular tasks are assigned to both units.

ALU – Arithmetic Logic Unit

The Arithmetic Logic Unit is made of two terms, arithmetic and logic. There are two primary functions that this unit performs.

1. Data is inserted through the input unit into the primary memory. Performs the basic arithmetical operations on it, like addition, subtraction, multiplication, and division. It performs all sorts of calculations required on the data. Then, it sends back data to the storage.
2. The unit is also responsible for performing logical operations like AND, OR, Equal to, Less than, etc. In addition to this, it conducts merging, sorting, and selection of the given data.

CU – Control Unit

The control unit as the name suggests is the controller of all the activities/tasks and operations. All this is performed inside the computer.

The memory unit sends a set of instructions to the control unit. Then the control unit in turn converts those instructions. After that these instructions are converted to control signals.

These control signals help in prioritizing and scheduling activities. Thus, the control unit coordinates the tasks inside the computer in sync with the input and output units.

Memory Unit

All the data that has to be processed or has been processed is stored in the memory unit. The memory unit acts as a hub of all the data. It transmits it to the required part of the computer whenever necessary.

The memory unit works in sync with the CPU. This helps in faster accessing and processing of the data. Thus, making tasks easier and quicker.

Types of Computer Memory

There are two types of computer memory-

Primary Memory

This type of memory cannot store a vast amount of data. Therefore, it is only used to store recent data. The data stored in this is temporary. It can get erased once the power is switched off. Therefore, is also called temporary memory or main memory.

RAM stands for Random Access Memory. It is an example of primary memory. This memory is directly accessible by the CPU. It is used for reading and writing purposes. For data to be processed, it has to be first transferred to the RAM and then to the CPU.

Secondary Memory

As explained above, the primary memory stores temporary data. Thus it cannot be accessed in the future. For permanent storage purposes, secondary memory is used. It is also called permanent memory or auxiliary memory. The hard disk is an example of secondary memory. Even in a power failure data does not get erased easily.

1.

Output

There is nothing to be amazed by what the output unit is used for. All the information sent to the computer once processed is received by the user through the output unit. Devices like printers, monitors, projectors, etc. all come under the output unit.

The output unit displays the data either in the form of a soft copy or a hard copy. The printer is for the hard copy. The monitor is for the display. The output unit accepts the data in binary form from the computer. It then converts it into a readable form for the user.

What is Registers and types of registers in CPU.

Registers are like the VIP lounges within the CPU. They're extremely fast, small storage locations where the CPU can keep the data it's actively working on. Think of them as the CPU's personal scratchpad, where it holds information temporarily to perform calculations, move data around, and execute instructions.

Why are Registers Important?

- **Speed:** Registers are the fastest type of memory in a computer. Accessing data in registers is significantly faster than accessing data in main memory (RAM). This speed boost is crucial for efficient processing.
- **Efficiency:** By keeping frequently used data and instructions in registers, the CPU can minimize its interactions with slower main memory, leading to faster program execution.

Types of Registers in a CPU

CPUs have various types of registers, each with a specialized role:

1. **General-Purpose Registers (GPRs):** These are the workhorses of the CPU. They can be used to store operands for arithmetic and logical operations, as well as addresses for accessing data in memory.

2. **Accumulator (ACC):** This register is traditionally used to store intermediate results during arithmetic and logical operations. In some architectures, it plays a central role in calculations.
3. **Memory Address Register (MAR):** This register holds the memory address of the data that the CPU wants to read or write. It's like telling the memory, "I need the data at this location!"
4. **Memory Data Register (MDR):** This register holds the actual data being read from or written to memory. It's like the messenger carrying the data between the CPU and memory.
5. **Program Counter (PC):** This register keeps track of the address of the next instruction to be executed. It ensures that the CPU executes instructions in the correct sequence.
6. **Instruction Register (IR):** This register holds the current instruction that the CPU is decoding and executing. It's like the CPU's instruction manual for the current operation.
7. **Stack Pointer (SP):** This register points to the top of the stack, which is a special area of memory used for temporary storage of data, function calls, and other operations.
8. **Status Register (SR) or Flags Register:** This register contains flags that indicate the status of the CPU or the results of operations. These flags can be used to control program flow or handle errors.

Key Takeaways

- Registers are small, high-speed storage locations within the CPU.
- They hold data and instructions that the CPU is actively using.
- Different types of registers serve specific purposes in the CPU's operations.
- Registers are essential for efficient program execution.

List out the advantages of micro programmed control unit.

Microprogrammed control units offer several key advantages, making them a popular choice in CPU design:

1. **Simpler Design and Implementation:** Compared to hardwired control units, microprogrammed control is generally easier to design and implement, especially for complex instruction sets. The control logic is more structured and less prone to errors.
2. **Flexibility and Adaptability:** Microprogrammed control is highly flexible. Changes to the instruction set or control logic can be made by simply modifying the microprogram stored in control memory. This eliminates the need for extensive hardware redesign.
3. **Ease of Modification and Debugging:** Modifying the control sequence is straightforward. Debugging is also easier as errors can be traced by examining the microprogram.

4. **Regularity and Modularity:** The microprogram structure lends itself to a more regular and modular design, which simplifies maintenance and updates.
5. **Lower Cost (for complex ISAs):** For processors with complex instruction sets (CISCs), microprogrammed control can be more cost-effective than hardwired control, as the hardware complexity is reduced.
6. **Support for Complex Instructions:** Microprogrammed control makes it easier to implement complex instructions that require multiple control signals and timing sequences.
7. **Diagnostic Capabilities:** Microprograms can be designed to include diagnostic routines that can be used to test and troubleshoot the control unit.
8. **Emulation:** Microprogrammed control can be used to emulate the instruction set of another computer, allowing a system to run software designed for a different architecture.
9. **Firmware Updates:** In some systems, the microprogram is stored in writable control store (WCS), allowing for field updates or bug fixes to the control unit's behavior. This is like firmware updates for other hardware components.

List the differences between cache and associative memory

Both cache memory and associative memory are used to speed up data access in computer systems, but they serve different purposes and operate in distinct ways. Here's a breakdown of their key differences:

Cache Memory

- **Purpose:** To store frequently accessed data and instructions, bringing them closer to the CPU for faster retrieval. It exploits the principle of locality of reference, where programs tend to reuse the same data and instructions repeatedly.
- **Access Method:** Data is accessed using its memory address. The CPU provides the address, and the cache checks if it holds the corresponding data.
- **Organization:** Cache memory is typically organized into sets and lines. Each set can hold multiple lines, and each line stores a block of data. Different mapping techniques (direct-mapped, set-associative, fully associative) determine how memory blocks are placed in the cache.
- **Search:** When the CPU requests data, the cache controller searches the cache based on the memory address. This involves comparing the address with tags associated with each cache line.
- **Implementation:** Cache memory is usually implemented using SRAM (Static RAM) for its speed.

Associative Memory (CAM)

- **Purpose:** To search for data based on its content rather than its address. It's also known as Content-Addressable Memory (CAM).
- **Access Method:** Data is accessed by providing a part or all of the data itself. The memory searches for a match and returns the location or associated information.
- **Organization:** Associative memory stores data along with tags or keys that describe the content. It allows for parallel searching of all stored data.
- **Search:** When the CPU provides data to search for, all locations in the associative memory are compared simultaneously. If a match is found, the corresponding location is identified.
- **Implementation:** Associative memory is often implemented using specialized hardware that allows for parallel comparisons.

Key Differences Summarized

Feature	Cache Memory	Associative Memory
Purpose	Store frequently used data	Search for data based on content
Access Method	Memory address	Data content
Organization	Sets and lines	Data with tags/keys
Search	Based on address	Parallel comparison of content
Speed	Fast	Very fast for content search
Cost	Relatively less expensive	More expensive
Use Cases	CPU caches, memory hierarchies	Look-up tables, pattern matching, network routers

In essence:

- Cache memory is like a quick-access storage for frequently used items, organized by their location (address).
- Associative memory is like a search engine that finds items based on what they are (content), regardless of their location.

While both enhance performance, they address different needs. Cache memory speeds up general data access, while associative memory excels at rapid content-based searches.

Discuss about nano programming in detail.

Nano programming is a technique used in microprogrammed control units to optimize the size of the control memory and improve the efficiency of control signal generation. It involves a two-level control memory structure, consisting of a micro control memory and a nano control memory.

Microprogrammed Control Units

Before diving into nano programming, let's briefly recap microprogrammed control units. In these units, control signals are generated by executing a microprogram stored in a control memory. Each microinstruction in the microprogram specifies a set of control signals to be activated for a particular step in the execution of a machine instruction.

Need for Nano Programming

Microprogrammed control units can be implemented using either horizontal or vertical microinstructions:

- **Horizontal Microinstructions:** Each bit in the microinstruction directly corresponds to a control signal. This allows for the simultaneous activation of multiple control signals, but it results in wide microinstructions and a large control memory.
- **Vertical Microinstructions:** Microinstructions are encoded, and a decoder is used to generate the control signals. This reduces the width of the microinstructions and the size of the control memory, but it limits the number of control signals that can be activated simultaneously.

Both approaches have their trade-offs. Nano programming aims to combine the advantages of both while mitigating their disadvantages.

Nano Programming Technique

Nano programming employs a two-level control memory hierarchy:

1. **Micro Control Memory (μCM):** Stores microinstructions, which are typically vertical microinstructions. These microinstructions are fetched and decoded to determine the type of operation to be performed.
2. **Nano Control Memory (nCM):** Stores nano instructions, which are horizontal microinstructions. Based on the decoded microinstruction, the appropriate nano instruction is fetched from the nano control memory. This nano instruction directly generates the required control signals.

How it Works

1. A machine instruction is fetched from main memory and loaded into the Instruction Register (IR).
2. The opcode of the instruction is used to determine the starting address of its microprogram in the micro control memory.
3. Microinstructions are fetched from the micro control memory and decoded.
4. The decoded microinstruction points to the starting address of the corresponding nano program in the nano control memory.
5. Nano instructions are fetched from the nano control memory and directly generate the control signals.

Advantages of Nano Programming

- **Reduced Control Memory Size:** By using a combination of vertical and horizontal microinstructions, nano programming reduces the overall size of the control memory compared to using only horizontal microinstructions.
- **Increased Control Signal Parallelism:** Nano instructions, being horizontal microinstructions, allow for the simultaneous activation of multiple control signals, providing greater parallelism than vertical microinstructions.
- **Improved Performance:** The combination of reduced memory size and increased parallelism can lead to improved performance compared to using only vertical microinstructions.
- **Greater Design Flexibility:** Nano programming provides greater flexibility in designing the control unit, allowing for a balance between control memory size and control signal parallelism.

Disadvantages of Nano Programming

- **Increased Memory Access Time:** The two-level memory structure in nano programming requires additional memory accesses, which can increase the overall memory access time.
- **Increased Complexity:** Implementing nano programming adds complexity to the control unit design, requiring additional hardware for the two-level memory structure and the associated control logic.

In Summary

Nano programming is a technique that combines the advantages of horizontal and vertical microinstructions to optimize the size of the control memory and improve the efficiency of control signal generation in microprogrammed control units. While it introduces some complexity and potential performance overhead, it offers a good balance between memory size, parallelism, and design flexibility.

SET 2 Part- C

Explain about the instruction set and its types?

The instruction set is the complete collection of instructions that a CPU can understand and execute. It's like the vocabulary of the CPU, defining all the operations it can perform. Each instruction tells the CPU to do something specific, like adding numbers, moving data, or making decisions.

Key Components of an Instruction

An instruction typically consists of:

- **Opcode:** This is the "verb" of the instruction, specifying the operation to be performed (e.g., ADD, SUB, LOAD, STORE).
- **Operands:** These are the "nouns" of the instruction, specifying the data or memory locations to be used in the operation.

Types of Instructions

Instruction sets can be categorized in various ways, but here are some common classifications:

1. **Data Transfer Instructions:** These instructions move data between registers, memory locations, and I/O devices. Examples include:
    - **LOAD:** Copies data from memory to a register.
    - **STORE:** Copies data from a register to memory.
    - **MOVE:** Copies data between registers or between memory locations.
    - **INPUT/OUTPUT:** Transfers data between the CPU and I/O devices.
2. **Arithmetic and Logical Instructions:** These instructions perform arithmetic and logical operations on data. Examples include:
    - **ADD, SUB, MUL, DIV:** Perform arithmetic operations.
    - **AND, OR, NOT, XOR:** Perform logical operations.
    - **SHIFTS, ROTATES:** Manipulate bits within data.
3. **Control Flow Instructions:** These instructions control the order in which instructions are executed. Examples include:
    - **JUMP:** Transfers program execution to a different address.
    - **BRANCH:** Conditionally jumps to a different address based on a condition.
    - **CALL, RETURN:** Used for calling and returning from subroutines.
4. **Other Instructions:** There are many other types of instructions, such as:

- **System Calls:** Request services from the operating system.
- **Specialized Instructions:** Perform specific tasks related to multimedia, cryptography, or other domains.

Types of Instruction Set Architectures (ISAs)

Instruction sets can also be categorized based on their overall design philosophy:

1. **Reduced Instruction Set Computing (RISC):**
    - Emphasizes simpler instructions that execute quickly.
    - Uses a large number of registers.
    - Relies on compilers to optimize code.
    - Examples: ARM, MIPS.
2. **Complex Instruction Set Computing (CISC):**
    - Features complex instructions that can perform multiple operations.
    - Uses fewer registers than RISC.
    - Puts more burden on the hardware for instruction decoding.
    - Examples: x86.
3. **Other ISAs:** There are other architectural styles, such as:
    - **Minimal Instruction Set Computing (MISC):** Uses a very small number of basic instructions.
    - **Very Long Instruction Word (VLIW):** Packs multiple instructions into a single word for parallel execution.
    - **Explicitly Parallel Instruction Computing (EPIC):** Designed for parallel processing.

Factors Affecting Instruction Set Design

The design of an instruction set is influenced by various factors, such as:

- **Target Applications:** The types of programs the computer will run.
- **Technology:** The capabilities of the underlying hardware.
- **Cost:** The cost of implementing the instruction set.
- **Performance:** The desired performance of the computer.
- **Software Compatibility:** The need to run existing software.

The instruction set is a fundamental aspect of computer architecture, shaping the capabilities and performance of a CPU. It's the language that programmers use to communicate with the hardware, and its design has a profound impact on the efficiency and effectiveness of software development.

Explain the different types of data representation?

Data representation is how information is encoded and stored within a computer system.[1] It's fundamental to how computers process and manipulate data, and there are several ways to represent different types of information.[2] Here's a breakdown of common data representation methods:

1. Number Systems

- **Binary (Base-2):** The foundation of digital computing.[3] Uses only two digits: 0 and 1. All data within a computer is ultimately represented in binary form.[4]
- **Octal (Base-8):**[5] Uses eight digits: 0-7.[6] Historically used as a more compact way to represent binary data.[7]
- **Decimal (Base-10):** The number system we use in everyday life, with digits 0-9.[8]
- **Hexadecimal (Base-16):** Uses sixteen symbols: 0-9 and A-F (where A=10, B=11, etc.). Often used in programming and digital systems as a shorthand for binary.[9]

2. Integer Representation

- **Unsigned Integers:** Represent positive whole numbers.[10] The number of bits used determines the range of values that can be represented.
- **Signed Integers:** Represent both positive and negative whole numbers.[11] Common methods include:
    - **Sign Bit:** The leftmost bit indicates the sign (0 for positive, 1 for negative).[12]
    - **One's Complement:** Negative numbers are represented by inverting all the bits of the positive equivalent.[13]
    - **Two's Complement:** The most common method for representing signed integers. Negative numbers are formed by inverting all the bits and adding 1 to the result.[14]

3. Floating-Point Representation

- Used to represent numbers with fractional parts or very large/small numbers.
- Based on scientific notation, with a mantissa (significant digits) and an exponent (scale factor).[15]
- Common standards include IEEE 754, which defines single-precision (32-bit) and double-precision (64-bit) formats.[16]

4. Character Encoding

- Represents text characters (letters, numbers, symbols) using numerical codes.[17]
- **ASCII:** A 7-bit encoding that represents 128 characters, including English letters, numbers, and punctuation marks.[18]
- **Extended ASCII:** An 8-bit encoding that extends ASCII to include additional characters.[19]
- **Unicode:** A universal character encoding standard that can represent virtually all characters from all writing systems around the world.[20] Common encodings include UTF-8 and UTF-16.[21]

5. Logical Values

- Represent truth values: true or false.
- Often represented by 1 (true) and 0 (false).

6. Images

- Images are represented as a collection of pixels, each with color information.[22]
- **Raster Graphics:** Images are stored as a grid of pixels.[23] Common formats include JPEG, PNG, GIF.
- **Vector Graphics:** Images are represented by mathematical equations that define lines, curves, and shapes.[24] Common formats include SVG.

7. Audio

- Sound is represented as a series of samples that capture the amplitude of the sound wave at different points in time.[25]
- Common formats include MP3, WAV, FLAC.[26]

8. Video

- Video is a sequence of images (frames) displayed over time, along with audio.[27]
- Common formats include MP4, AVI, MOV.[28]

Key Considerations

- **Data Type:** The type of data being represented (number, text, image, etc.) determines the appropriate representation method.
- **Range and Precision:** The number of bits used affects the range of values that can be represented and the precision of the representation.
- **Storage Requirements:** Different representation methods have different storage requirements.
- **Processing Efficiency:** The choice of representation can impact the efficiency of data processing.

Data representation is a crucial aspect of computer science, as it determines how information is stored, manipulated, and exchanged within computer systems.[29] Understanding these different methods is essential for working with computers and digital data.

Define pipeline hazard. Explain its types in detail.

In a pipelined processor, multiple instructions are overlapped in execution, similar to an assembly line. While this boosts performance, it can lead to situations where instructions interfere with each other, causing delays or incorrect results. These situations are called **pipeline hazards**.

Think of it like this: Imagine a car assembly line where workers are installing engines, wheels, and interiors simultaneously on different cars. If one worker needs a part that's not ready yet because another worker is still using it, that worker has to wait, causing a delay in the whole line. That's similar to a pipeline hazard.

There are three main types of pipeline hazards:

1. Structural Hazards

- **What it is:** Occurs when multiple instructions try to access the same hardware resource at the same time. This could be memory, an ALU (Arithmetic Logic Unit), or a register.
- **Example:** Imagine two workers on the car assembly line needing the same wrench at the same time. One has to wait. Similarly, if two instructions need to access memory in the same clock cycle, one will have to wait, causing a stall in the pipeline.
- **Solution:** Increase the number of resources (e.g., have multiple memory units or ALUs) or schedule instructions so that they don't conflict.

2. Data Hazards

- **What it is:** Arise when an instruction depends on the result of a previous instruction that is still in the pipeline. The instruction might try to use data that is not yet available.
- **Types:**
    - **Read After Write (RAW) / True Dependency:** An instruction tries to read a value before a previous instruction has written it. (Like a worker trying to use an engine before it's been fully installed.)
    - **Write After Read (WAR) / Anti-dependency:** An instruction writes to a location before a previous instruction has read from it. (Less common, but imagine a worker changing the car's interior before another worker has finished using a tool inside.)
    - **Write After Write (WAW) / Output Dependency:** Two instructions both try to write to the same location, and the order matters. (Imagine two workers trying to install the same part, and the order in which they do it affects the final result.)

- **Solutions:**
    - **Forwarding (Bypassing):** The result of an instruction is directly passed to the next instruction that needs it, even before it's written to memory. (Like passing the tool directly from one worker to the next.)
    - **Stalling (Inserting Bubbles):** The pipeline is stalled until the data is available. (Like making the worker wait for the part to be ready.)
    - **Compiler Scheduling:** The compiler reorders instructions to minimize dependencies.

3. Control Hazards

- **What it is:** Occur due to branch instructions, which change the flow of control in the program. The pipeline might fetch instructions along the wrong path before the branch condition is evaluated.
- **Example:** Imagine a worker coming to a fork in the assembly line. They need to know which way to go, but the decision depends on something that's still being worked on further down the line.
- **Solutions:**
    - **Branch Prediction:** The processor tries to guess which way the branch will go and fetches instructions along the predicted path. (Like the worker guessing which way to go at the fork.)
    - **Delayed Branching:** The branch instruction is delayed by a few instructions. The instructions immediately following the branch are always executed, regardless of the branch outcome. (Like the worker continuing to do some tasks before reaching the fork.)
    - **Branch Target Buffer:** A cache that stores the target addresses of recently taken branches.

Why are Hazards Important?

Pipeline hazards can reduce the efficiency of a pipelined processor. They introduce stalls or require complex mechanisms to resolve, which can impact performance. Understanding hazards and how to mitigate them is crucial for designing and optimizing pipelined processors.

List out the differences between RAM and ROM.

Feature	RAM (Random Access Memory)	ROM (Read-Only Memory)
Full Name	Random Access Memory	Read-Only Memory
Volatility	Volatile	Non-volatile
Data Storage	Temporary	Permanent
Read/Write	Read and Write	Read-only (typically)
Speed	Fast	Slower than RAM
Cost	More expensive per bit	Less expensive per bit
Capacity	Typically larger	Typically smaller
Use in System	Working memory for programs and data	Stores critical startup instructions (BIOS), firmware
Modifiability	Data can be easily changed	Data is usually fixed at manufacturing

In simpler terms:

- **RAM is like your desk:** You can put things on it, take things off, and rearrange things as needed.[1] But when you turn off the light (power off the computer), everything on the desk disappears.
- **ROM is like a printed manual:** It contains information that you can read, but you can't easily change what's written in it.[2] This information is always there, even when the power is off.

Why these differences matter:

- **RAM's volatility:** Makes it ideal for holding the programs and data that the CPU is actively using.[3] This allows for fast access and modification, which is essential for running software and performing tasks.[4]

- **ROM's non-volatility:** Makes it perfect for storing crucial instructions that the computer needs to start up (like the BIOS) or for holding firmware in devices like printers and routers.[5] These instructions need to be preserved even when the device is powered off.

Important Note: While the name suggests "Read-Only," there are some types of ROM that can be erased and reprogrammed (like EEPROM).[6] However, this is usually done infrequently and requires special procedures.

With a neat schematic DMA controller and its mode of data transfer

DMA Controller in Computer Architecture

DMA Controller is a type of control unit that works as an interface for the data bus and the I/O Devices. As mentioned, DMA Controller has the work of transferring the data without the intervention of the processors, processors can control the data transfer. DMA Controller also contains an address unit, which generates the address and selects an I/O device for the transfer of data. Here we are showing the block diagram of the DMA Controller.

Types of Direct Memory Access (DMA)

There are four popular types of DMA.

- Single-Ended DMA
- Dual-Ended DMA
- Arbitrated-Ended DMA
- Interleaved DMA

Single-Ended DMA: Single-Ended DMA Controllers operate by reading and writing from a single memory address. They are the simplest DMA.

Dual-Ended DMA: Dual-Ended DMA controllers can read and write from two memory addresses. Dual-ended DMA is more advanced than single-ended DMA.

Arbitrated-Ended DMA: Arbitrated-Ended DMA works by reading and writing to several memory addresses. It is more advanced than Dual-Ended DMA.

Interleaved DMA: Interleaved DMA are those DMA that read from one memory address and write from another memory address.

Working of DMA Controller

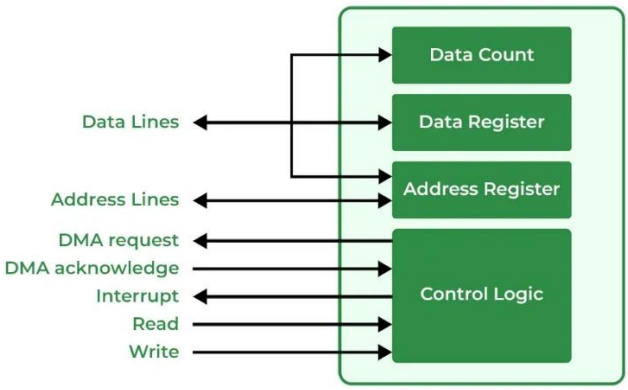
The DMA controller registers have three registers as follows.

- **Address register –** It contains the address to specify the desired location in memory.
- **Word count register –** It contains the number of words to be transferred.

- **Control register –** It specifies the transfer mode.

Note: All registers in the DMA appear to the CPU as I/O interface registers. Therefore, the CPU can both read and write into the DMA registers under program control via the data bus.

The figure below shows the block diagram of the DMA controller. The unit communicates with the CPU through the data bus and control lines. Through the use of the address bus and allowing the DMA and RS register to select inputs, the register within the DMA is chosen by the CPU. RD and WR are two-way inputs. When BG (bus grant) input is 0, the CPU can communicate with DMA registers. When BG (bus grant) input is 1, the CPU has relinquished the buses and DMA can communicate directly with the memory.



Explanation: The CPU initializes the DMA by sending the given information through the data bus.

- The starting address of the memory block where the data is available (to read) or where data are to be stored (to write).
- It also sends word count which is the number of words in the memory block to be read or written.
- Control to define the mode of transfer such as read or write.
- A control to begin the DMA transfer

Modes of Data Transfer in DMA

There are 3 modes of data transfer in DMA that are described below.

- **Burst Mode:** In Burst Mode, buses are handed over to the CPU by the DMA if the whole data is completely transferred, not before that.
- **Cycle Stealing Mode:** In Cycle Stealing Mode, buses are handed over to the CPU by the DMA after the transfer of each byte. Continuous request for bus control is generated by this Data Transfer Mode. It works more easily for higher-priority tasks.
- **Transparent Mode:** Transparent Mode in DMA does not require any bus in the transfer of the data as it works when the CPU is executing the transaction.